מסלול טיולים אפקה 2026 🌍

מגישות: מעין שני ורותם גלבוע

- **טכנולוגיות מרכזיות:** Next.js (Frontend), Express.js (Backend), MongoDB (Database), Groq AI (LLM).
- **קישור למאגר הקוד (GitHub):**
- https://github.com/RotemGilboa2/Afeka-Travel-Planner-2026
- https://github.com/Maayan-Shani/Afeka-Travel-Planner-2026
- **קישור לאתר:**
- Local Environment : http://localhost:3000

Made with GAMMA

בעיות ידועות (Known Bugs & Limitations)

הגבלת בקשות (Rate Limiting) בשירותי מפות

מכיוון שאנו משתמשות ב-API חינמי של OpenStreetMap (Nominatim), לעיתים יש עומס או חסימה זמנית. *טיפול שלנו:* הוספנו מנגנון Timeout (AbortController) והגדרת User-Agent ייעודי למניעת קריסת הדפדפן.

חריגות פורמט מה-LLM

לעיתים נדירות מודל ה-AI (Llama 3.1) מחזיר טקסט חופשי בנוסף ל-JSON המבוקש. *טיפול שלנו:* הפעלנו response_format: { type: "json_object" } בשרת ועטפנו את פענוח הטקסט ב-try-catch בקליינט.

תמונות ערים קטנות

משיכת התמונה מוויקיפדיה נכשלת אם לעיר אין ערך מקושר עם תמונה ראשית. במקרה כזה תופיע חלונית "אין תמונה זמינה".

Made with GAMMA

מבנה ארכיטקטוני (Architecture)

צד לקוח (Next.js - Frontend)

- אחראי על ה-(React) UI/UX.
- מנהל את המפה (React-Leaflet).
- השתמשנו ב **Middleware:** שומר על נתיבים פרטיים ומבצע **Silent Refresh** (ריענון שקט לאסימון מול השרת) מבלי שהמשתמש ירגיש.

צד שרת (Express.js - Backend)

- מנהל את פניות ה-API ומסתיר את מפתחות ההצפנה (API Keys).
- ה **Auth:** מנפק אסימוני JWT מוגנים (Access + Refresh). סיסמאות מוצפנות עם Salt דרך Bcrypt.
- תקשורת מול ה-(Groq) LLM להפקת המלצות.

מסד נתונים (MongoDB)

- שומר משתמשים וסיסמאות מגובבות (Hashed).
- שומר את היסטוריית המסלולים (קואורדינטות, מרחקים, והמלצות ה-AI שחולצו).

שירותים חיצוניים (3rd Party APIs)

- מפות וניווט: OSRM + Nominatim.
- מזג אוויר: Open-Meteo.

Made with GAMMA

קטעי קוד חשובים (דגימות)

דגימה 1: "Silent Refresh" ו-Middleware

בדיקה לפני כניסה: ה-Middleware מוודא שהאסימון (Access Token) בתוקף לפני שניתנת גישה לדפים המוגנים.

ריענון נסתר ("Silent"): אם האסימון פג, נשלחת בקשת רקע שקופה לשרת (/refresh) במקום להחזיר את המשתמש לדף ההתחברות.

עדכון עוגיות (Cookies): השרת מנפק אסימונים חדשים, ואנחנו שותלות אותם מיד בדפדפן בעזרת NextResponse.next().

חוויה חלקה ורציפה: המשתמש מקבל הרשאה מחודשת מאחורי הקלעים, וממשיך לעבוד באתר מבלי לשים לב לפעולה.

בקובץ middleware.ts:

```
const refreshResponse = await
fetch${process.env.NEXT_PUBLIC_API_URL}/auth/refresh, {
    method: "POST",
    headers: { cookie: cookieHeader },
    cache: "no-store",
  });
  if (!refreshResponse.ok) {
    return NextResponse.redirect(new URL("/login", request.url));
  }
  const response = NextResponse.next();
  const anyHeaders = refreshResponse.headers as any;
  const setCookies: string[] = anyHeaders.getSetCookie?.() ?? [];
  for (const c of setCookies) {
    response.headers.append("set-cookie", c);
  }
  if (setCookies.length === 0) {
    const single = refreshResponse.headers.get("set-cookie");
    if (single) response.headers.append("set-cookie", single);
  }
  return response;
}
```

Made with GAMMA

קטעי קוד חשובים (דגימות)

דגימה 2: בניית מסלול טרק מעגלי (OSRM)

- **האתגר בניווט:** שירותי מפות מחזירים "0 ק"מ" כשמבקשים מנקודה מסוימת לחזור בדיוק אליה ללא יעד ביניים.
- **יעד ביניים (Waypoint):** הוספנו לוגיקה שמגרילה נקודת ציון רנדומלית ברדיוס של כ-2.5 ק"מ מנקודת המוצא.
- **בקשה משולשת:** ה-API של OSRM מקבל כעת 3 נקודות: עיר המוצא ← יעד הביניים ← עיר המוצא.
- **התוצאה:** מנוע הניווט מחשב מסלול ריאלי על שבילים קיימים (הלוך ושוב) ומייצר מעגל באורך של 5-10 ק"מ, כנדרש.

בקובץ PlanPage.tsx:

```
if (tripType === "trek") {

const radius = 2 + Math.random() * 1.5; const waypoint = generateRandomDestination(geo.lat, geo.lng, radius);

const start = ${geo.lng},${geo.lat}; const mid = ${waypoint.lng},${waypoint.lat}; const end = ${geo.lng},${geo.lat};

url = https://router.project-osrm.org/route/v1/${profile}/${start};${mid};${end}?overview=full&geometries=geojson; } else {
```

Made with GAMMA

קטעי קוד חשובים (דגימות)

דגימה 3: אינטגרציית ה-AI וה-Prompt

הזנת נתונים דינמיים: השרת מעביר ל-LLM את מרחקי האמת שחושבו מהמפה (dailyLines) כדי לקבל המלצות מדויקות לכל יום.

הנחיית פורמט קשיחה (Prompt Engineering): הגדרנו בפרומפט מבנה JSON מדויק שנדרש לחזור כפלט.

אכיפת אובייקט (JSON Mode): שימוש ב-response_format: { type: "json_object" } כדי להבטיח שהמודל לא יחזיר טקסט חופשי שיקריס את הלקוח.

מודל מתקדם ומהיר: השתמשנו במודל Llama 3.1 8B דרך פלטפורמת Groq כדי להבטיח זמני תגובה כמעט מיידיים למשתמש.

בקובץ index.js:

```
app.post("/ai/trip", requireAuth, async (req, res) => { try { const { city, tripType, days, daysPlan } = req.body; const dailyLines = Array.isArray(daysPlan) && daysPlan.length > 0 ? daysPlan.map((d) => Day ${d.dayIndex}: exactly ${Number(d.distanceKm).toFixed(2)} km).join("\n") : Days: ${days};

const prompt = `

You are a local travel expert. The user is going on a ${tripType} trip in ${city}. Specific route distances for each day: ${dailyLines}

Your task: For each day, suggest 2-3 specific local restaurants/cafes and 2-3 interesting attractions or viewpoints.

Return JSON ONLY in this format: { "title": "Culinary & Discovery Guide for ${city}", "summary": "A brief overview.", "days":[ { "day": 1, "distance": "...", "description": "RESTAURANTS: [...]. ATTRACTIONS: [...]" } ] } `;

const response = await groq.chat.completions.create({ model: "llama-3.1-8b-instant", messages: [{ role: "user", content: prompt }], temperature: 0.4, response_format: { type: "json_object" }, });

res.json({ ok: true, text: response.choices[0].message.content });
```

Made with GAMMA